

Instagram, 2010-style — SwiftUI

A small recreation of the original app: square photos only, a handful of vintage CoreImage filters standing in for X-Pro II / Lo-Fi / Earlybird / Hefe / Sutro / Toaster, a plain reverse-chronological feed (no algorithm), and nothing else — no Stories, DMs, video, or ads.

Setup

1. In Xcode: **File → New → Project → iOS App**, name it `InstagramClone2010`, interface **SwiftUI**, language **Swift**.
2. Delete the generated `ContentView.swift` and `InstagramClone2010App.swift`.
3. Drag the `Models`, `Views`, `Utilities` folders and the two root `.swift` files from this project into your Xcode project (check "Copy items if needed").
4. Add a Photo Library usage description: open your target's **Info** tab and add key `Privacy - Photo Library Usage Description ( NSPhotoLibraryUsageDescription )` with a value like "Used to pick photos to post."
5. Run on an iOS 16+ simulator or device (uses `PhotosPicker` and the `.onChange(of:)` two-closure API from iOS 16/17).

What's in here

- `Models/Post.swift` — a post: username, image, filter used, caption, timestamp
- `Models/FilterType.swift` — CoreImage recreations of the classic filter pack
- `Utilities/ImageProcessing.swift` — center square-crop + filter rendering
- `Utilities/SkeuomorphicStyle.swift` — the "real materials" look: procedurally drawn leather grain, wood grain, cream stitching, a glossy chrome shutter button, polaroid-style photo frames, and a script wordmark font
- `PostStore.swift` — in-memory, `@Published`, sorted newest-first
- `Views/RootTabView.swift` — Feed / Camera / Profile tabs, wood-grain tab bar
- `Views/FeedView.swift` — leather nav bar, script "Instagram" wordmark, stitched/polaroid-framed feed

- `Views/NewPostFlow.swift` — leather-backed capture screen with a glossy circular shutter button → filter strip → caption → share

About the skeuomorphic pass

This recreates the *style* of the 2010 iOS look (stitched leather, wood grain, glossy chrome, hand-lettered script logotype) using original SwiftUI/CoreGraphics-drawn textures — not copied Instagram assets or their actual logo file. The leather and wood grain are procedurally drawn in `SkeuomorphicStyle.swift` (`Canvas` + a seeded RNG) rather than bitmap textures, so there's nothing to source or license.

The wordmark uses **Snell Roundhand**, a script font bundled with iOS, as a stand-in for the original's custom script logotype — it's not pixel-identical but reads the same "handwritten label" way. If it's ever unavailable it falls back to a bold serif italic automatically.

Connecting your real Instagram account

The Profile tab has a "Connect Instagram" button that runs a real OAuth login against Meta's **Instagram API with Instagram Login** and, once connected, an "Import My Photos" button that pulls your own posts into the feed (square- cropped, no filter re-applied — they come in as you originally posted them).

This only works for accounts you control, and only reads your own posts — there's no way to browse other people's feeds or anything follow-based; Meta doesn't offer that to third-party apps anymore (see caveats below).

If you connected before this update, disconnect and reconnect once — the login now requests an additional scope (`instagram_business_content_publish`) for the publish flow below, and a token granted under the old scope won't have it.

One-time setup on Meta's side

1. **Convert your Instagram account to Business or Creator.** This API only works with professional accounts — Settings → Account type in the Instagram app.
2. **Create a Meta developer app:** go to developers.facebook.com, create an account if needed, then **My Apps → Create App** → choose a type that supports Instagram → give it any name.
3. **Add the Instagram product:** in your app's dashboard, add **Instagram → API setup with Instagram login**.
4. Under **Business login settings**, add your **redirect URI** (see below) and note your

Instagram App ID and Instagram App Secret.

5. Under **Roles** → **Instagram testers**, add your own Instagram account as a tester, and accept the invite from inside the Instagram app (Settings → Apps and websites → Tester invites). This lets you use the API on your own account without submitting for Meta's app review.

The redirect page (required — Meta won't redirect to a custom URL scheme)

Meta's OAuth flow only redirects to an **https** URL, not directly to an app. The standard workaround is a tiny static page that immediately forwards into the app via its custom URL scheme. Host this anywhere static (GitHub Pages is free and easy):

```
<!-- redirect page, e.g. https://your-username.github.io/ig-redirect/index.html -  
<script>  
    window.location.href = "instaclone2010://callback" + window.location.search;  
</script>
```

Use that page's URL as the **redirect URI** in step 4 above, and as

`InstagramConfig.redirectURI` below.

Configure the app

Open `InstagramConfig.swift` and fill in:

- `appID` — your Instagram App ID
- `appSecret` — your Instagram App Secret
- `redirectURI` — your hosted redirect page's exact URL

Then register the custom URL scheme so iOS routes the redirect back into the app: in Xcode, select your target → **Info** tab → **URL Types** → add one with **URL Schemes** set to `instaclone2010` (or whatever you set `callbackScheme` to).

Security note

`InstagramAuthManager` stores the access token in `UserDefaults` and the app embeds your `appSecret` directly in the client for simplicity. That's fine for a personal build you run on your own device, but **don't ship this to the App Store or hand it to other people as-is** — an app secret baked into a distributed binary can be extracted. A real product would do the code-for- token exchange on a server you control instead, keeping the secret off the device entirely.

What this can't do, even connected

- Browse other people's photos or a following-based feed — Meta doesn't expose that to third-party apps (the old API that allowed it, Basic Display, was shut down in December 2024)
- Work with a plain personal (non-Business/Creator) account
- More than 25 API-published posts per account per rolling 24 hours (Meta's cap)

Actually publishing to your real Instagram

The "Also post to my real Instagram" toggle in the New Post screen does a real publish, but Instagram's publish endpoint doesn't accept a file upload from your phone — it downloads the image itself from a URL you give it. So publishing is really three steps: **upload the filtered photo somewhere public → tell Instagram that URL → wait for it to fetch/validate → publish.**

This project uses **Firebase Storage** as that "somewhere public."

Firebase setup

The app runs fine without any of this — `InstagramClone2010App` only calls `FirebaseApp.configure()` if it finds a `GoogleService-Info.plist` in the bundle, and `FirebaseUploader` throws a friendly error (rather than crashing) if you try to publish before that's set up. So you can build and use everything else — feed, filters, Instagram login/read/import — and add this later, whenever you're ready to actually publish.

One thing that *can't* be deferred: the **Swift Package Manager dependencies**

(`FirebaseStorage`, `FirebaseAuth`) still need to be added for the project to compile at all — that's a build-time requirement, not a runtime config. Adding the packages ≠ setting up a Firebase project; you can add the packages now and do everything below whenever you're ready.

1. Go to console.firebase.google.com, create a project (free Spark plan is enough for personal use).
2. **Build → Storage** → get started, and pick a location.
3. **Project settings → your apps → add an iOS app.** Use your Xcode project's bundle ID. Download the generated `GoogleService-Info.plist` and drag it into the Xcode project (check "Copy items if needed").
4. In Xcode: **File → Add Package Dependencies →**

<https://github.com/firebase/firebase-ios-sdk> → add the **FirestoreStorage** and **FirestoreAuth** products to your target.

5. **Build → Authentication → get started → Sign-in method → enable Anonymous.** The app signs in anonymously before each upload so writes require *some* auth, even though nobody logs into an account for it.

6. **Storage → Rules**, and set:

```
rules_version = '2';
service firebase.storage {
  match /b/{bucket}/o {
    match /posts/{file} {
      allow read: if true;           // Instagram's servers need public read
      allow write: if request.auth != null; // but only signed-in (anonymous)
    }
  }
}
```

That's it — no server of your own to run. Firebase hosts the uploaded JPEG, Instagram's container step fetches it from there, and once published you can delete it from Storage if you don't want to keep hosting it (the post itself lives on Instagram from that point on, independent of the file).

Why the extra Firebase project instead of your own server

You could swap `FirebaseUploader.swift` for any host that gives back a public HTTPS URL for an uploaded file (S3 + presigned URLs, your own backend, etc.) — Firebase Storage is just the least setup for a personal, single-user build.

`InstagramAPIClient.createMediaContainer` only cares that the `image_url` it's given is publicly fetchable; it doesn't know or care where it's hosted.

Notes / things a real build would need

- Posts are in-memory only (lost on relaunch) — swap `PostStore` for Core Data, `SwiftData`, or a backend if you want persistence.
- The feed comes pre-populated with 6 placeholder posts (fake usernames, gradient-and-icon "photos" generated in code — see `SeedData.swift`) so it isn't empty on first launch. They're not real stock photography — nothing to license or attribute. They only appear when the store is empty, so they won't reappear once you've posted anything real.

- No camera capture wired up, just the photo library picker — add `UIImagePickerController` (or `AVFoundation`) for a live camera.
- No following, likes, or comments that actually do anything — the original didn't have most of those on day one either.
- Imported photos land with the `.normal` filter (they already look like whatever you originally posted) — nothing stops you from re-filtering them manually if you want the vintage look applied on top.